	Cairo University **Faculty of Graduate Studies for Statistical Research** **Software Engineering Program**	

Professional Master's Graduation Project

Software Engineering Program – Coursework Track

Project Title

MeetFlow AI: An AI-Powered Mobile Application for Automated Meeting Documentation and Task Tracking

Software engineer

Dr Mager mamdouh

Student Name	
Student ID	202401385
Program	Master of Software Engineering
Track	Coursework Track
Supervisor Name	To be confirmed by the department
Academic Year	2025 / 2026
Submission Date	June 7, 2026

Submitted in partial fulfillment of the requirements for the Professional Master's Degree in Software Engineering.

Table of Contents

This table of contents summarizes the main report sections and page numbers in the final formatted document.

Abstract

Meetings are a primary medium for decision-making in academic, professional, and business environments, yet the knowledge produced during them is frequently lost. Notes are taken inconsistently, action items are forgotten, and reviewing a recording to recover decisions is slow and error-prone. This project addresses the problem of capturing meeting outcomes in a structured, reliable, and reusable form.

The proposed solution, **MeetFlow AI**, is a cross-platform mobile application that automatically transforms meeting audio, video, or publicly accessible transcript/recording links into structured documentation. Authenticated users upload a recording or paste a link, and the system generates a concise summary, a detailed summary, minutes of meeting, extracted decisions, and a list of actionable tasks. Generated tasks are persisted and can be tracked after the meeting, giving users a single place to follow up on commitments.

The system is implemented in Flutter and Dart using a feature-first clean architecture. It uses Firebase Authentication for secure sign-in and Cloud Firestore for per-user data persistence, with Riverpod for state management and GoRouter for navigation. The artificial-intelligence capability is provided by Google's Gemini multimodal large language model, accessed through Firebase AI Logic so that the API key is never embedded in the client. The model is prompted to return a strict JSON schema and to preserve the original meeting language (for example, Arabic input yields Arabic output).

A working prototype was implemented and validated against the project objectives. The application demonstrates a complete end-to-end workflow: authentication, media import, AI processing, structured storage, and post-meeting task tracking. Manual validation confirmed that the core journeys work on the implemented Flutter/Firebase prototype, and the screenshots and evaluation summary in Chapters 5 and 7 provide evidence of the completed user-facing flows.

Keywords

Mobile Application Development, Flutter, Generative AI, Large Language Models, Gemini, Meeting Documentation, Firebase, Cloud Firestore, Task Management.

1. Introduction

1.1 Background

Meetings remain one of the most common knowledge-work activities across academia, industry, and government. They are where requirements are clarified, decisions are made, and responsibilities are assigned. However, the value of a meeting is realized only if its outcomes are captured and acted upon. In practice, much of this value is lost: attendees take fragmentary notes, recordings are rarely revisited, and the link between what was decided and who must act is weak.

In parallel, generative artificial intelligence has matured rapidly. Modern large language models (LLMs) can summarize long passages, extract entities, and follow structured output instructions. The latest generation of multimodal models, such as Google's Gemini, can additionally accept audio and video directly, removing the need for a separate speech-to-text stage. Combined with cloud platforms that offer authentication, databases, and managed AI access on free or low-cost tiers, it is now feasible for a single developer to build an intelligent, serverless mobile application. This project sits at the intersection of these trends: applying multimodal generative AI to the long-standing problem of meeting documentation, delivered through a modern cross-platform mobile client.

1.2 Project Motivation

The motivation for MeetFlow AI is both practical and technical. From a practical standpoint, professionals and students lose significant time manually writing minutes, and important action items routinely slip because they are never recorded in a trackable form. A tool that converts a raw recording into a clean summary, a list of decisions, and a set of trackable tasks directly addresses this pain.

From a technical and educational standpoint, the project is an opportunity to demonstrate sound software-engineering practice: a clean, layered architecture; secure handling of credentials and AI keys; reactive state management; and the integration of a third-party AI capability behind an abstraction that can later be replaced by a backend service. It also explores a genuinely useful capability of multimodal LLMs — processing audio/video directly — and the engineering required to make their output reliable through strict structured prompting and defensive parsing.

1.3 Project Objectives

The project pursues the following specific and measurable objectives:

1. **O1.** Provide secure user access — implement email/password registration, login, password reset, and persistent sessions using Firebase Authentication, with each user's data isolated from others.
2. **O2.** Enable flexible meeting import — allow users to upload audio/video files (MP3, WAV, M4A, MP4, MOV, up to 50 MB) or paste a public transcript/recording link as the input source.
3. **O3.** Generate structured documentation automatically — use a multimodal LLM to produce a short summary, detailed summary, minutes of meeting, extracted decisions, participants, follow-ups, and action items in a fixed JSON schema, preserving the original meeting language.
4. **O4.** Support post-meeting follow-through — persist generated meetings and tasks per user, present a meeting history and detail view, and allow action items to be tracked through pending, in-progress, and completed states.

1.4 Project Scope

In scope: a mobile application (Android and iOS via Flutter); email/password authentication; importing meetings from local audio/video files or public HTTP(S) links; AI generation of summaries, minutes, decisions, participants, follow-ups, and tasks; per-user persistence of meetings, decisions, and tasks; meeting history and details; task tracking; light/dark theming; and bilingual output (the AI preserves the meeting's language, validated primarily for English and Arabic).

Out of scope (for the MVP): real-time/live transcription during a meeting; automatically joining meetings as a bot on Zoom/Google Meet/Microsoft Teams (such invite links are detected and rejected with guidance); storing raw media files in the cloud (only metadata and generated results are stored); team collaboration and sharing across accounts; and a dedicated production backend. A backend proxy for the AI key is recommended as future work.

Assumptions and constraints: the user's device has internet connectivity; the meeting recording is reasonably audible; uploaded files respect the 50 MB inline-processing limit imposed by direct client-to-model requests; and the project relies on free or developer-tier quotas of Firebase and Gemini, so unlimited usage is not assumed.

1.5 Document Organization

- **Chapter 2 – Problem Definition:** defines the problem, stakeholders, the current (as-is) situation, its impact, and the requirements derived from it.
- **Chapter 3 – Existing Solution Approaches:** reviews comparable tools and techniques and identifies the gap this project fills.
- **Chapter 4 – Proposed Solution:** describes the solution, its rationale, key features, architecture, technology stack, and risks.
- **Chapter 5 – System Analysis and Design:** presents functional and non-functional requirements, use cases, the data model, UI design, and design diagrams.
- **Chapter 6 – Implementation:** details the development environment, implementation of the main modules, key design decisions, security, and deployment.
- **Chapter 7 – Testing and Evaluation:** describes the testing strategy, test cases, metrics, results, and validation against the objectives.
- **Chapter 8 – Discussion:** reflects on the problem status after the solution, benefits, remaining limitations, and lessons learned.
- **Chapter 9 – Conclusion and Future Work:** summarizes the project, its contributions, and proposed enhancements.

2. Problem Definition

2.1 Problem Statement

Organizations and individuals conduct frequent meetings but lack a fast, reliable, and low-cost way to convert those meetings into structured, actionable records. Manual minute-taking is inconsistent and labor-intensive; it distracts a participant from the discussion and still produces notes that vary in quality. Decisions and action items are often not recorded in a structured form, so accountability after the meeting is weak and follow-up depends on memory. Existing automated tools that solve part of this problem are typically subscription-based desktop/web services, are not optimized for a lightweight personal mobile workflow, and often handle non-English content (such as Arabic) poorly.

The core problem is therefore: how to automatically transform a raw meeting recording or transcript into accurate, structured documentation — summary, minutes, decisions, and trackable tasks — within a single, secure, low-cost mobile application.

2.2 Stakeholders and Users

The following stakeholders and system actors are affected by the problem:

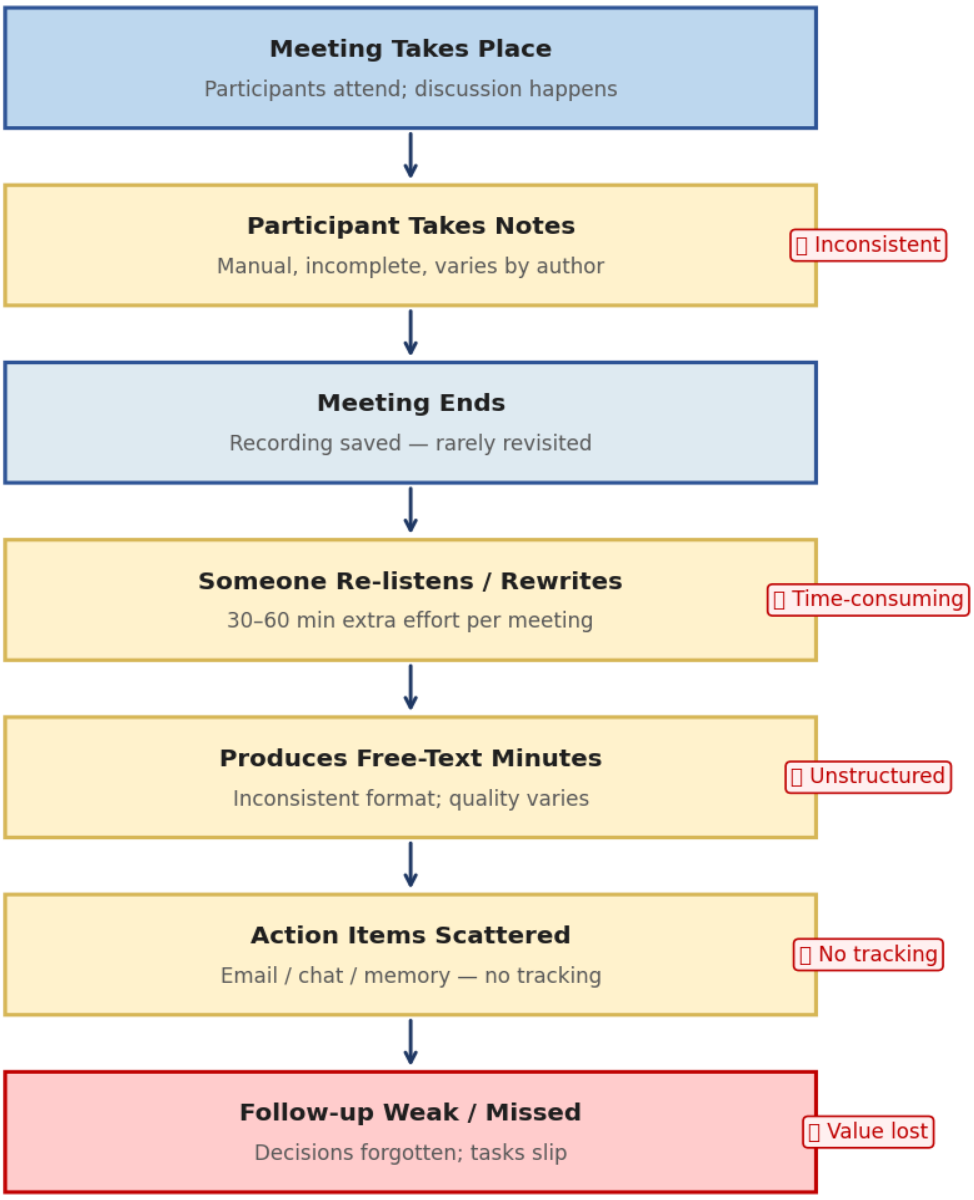
Stakeholder / Actor	Interest / Role
Individual professional	Primary user; needs quick minutes, decisions, and a personal task list after meetings.
Student / researcher	Captures lectures, supervision meetings, and study-group sessions into reviewable notes.
Team lead / project manager	Needs decisions and action items with owners to drive follow-up and accountability.
System (AI service)	The Gemini multimodal model that analyzes media and returns structured documentation.
System (Firebase)	Authentication and Cloud Firestore that authenticate users and persist their data.
Developer / maintainer	Builds and maintains the application; concerned with security, cost, and maintainability.

2.3 Current Situation / As-Is Process

Today, a typical meeting follow-up workflow is largely manual:

5. A participant takes free-form notes during the meeting, or the meeting is recorded for later review.
6. After the meeting, someone re-listens to the recording or rewrites their notes to produce minutes — a slow, error-prone task that is frequently skipped.
7. Decisions and action items, if captured at all, are scattered across notes, chat messages, or email and are not tracked in a consistent place.
8. Generic AI chat tools or transcription services may be used ad hoc, but they require manual copying, are not structured for meetings, and raise privacy and cost concerns.

Figure 2.1: As-Is Meeting Documentation Process (Manual)
Before MeetFlow AI



Problem: Meeting knowledge is lost and action items are not systematically tracked.

Figure 2.1: As-is meeting documentation process.

2.4 Problem Impact

Failing to solve this problem has measurable consequences across several dimensions:

- **Time and cost:** manually writing minutes for a one-hour meeting can take 30–60 minutes of skilled effort, repeated for every meeting.

- **Quality and accuracy:** hand-written minutes omit details and are inconsistent between authors and meetings.
- **Accountability:** when action items are not recorded with owners and status, commitments are forgotten and follow-up fails.
- **Accessibility and language:** non-English meetings (e.g., Arabic) are poorly served by many existing tools, excluding a large group of users.
- **Privacy and cost:** pasting sensitive meeting content into ad hoc or paid third-party services raises confidentiality and budget concerns.

2.5 Requirements Derived from the Problem

The problem definition leads directly to the following high-level requirements, refined in Chapter 5:

- **Functional:** secure per-user access; import of meeting audio/video files and public links; automatic generation of structured documentation (summary, minutes, decisions, participants, follow-ups, tasks); persistence and history of meetings; and tracking of action items.
- **Non-functional:** security of credentials and the AI key; per-user data isolation; usability (few steps to a result); responsive feedback during processing; reliability and graceful error handling; bilingual (Arabic/English) output; and low operating cost using free/managed tiers.

3. Existing Solution Approaches

3.1 Literature / Market Review

A range of commercial tools and platforms already address parts of the meeting-documentation problem. Representative examples include:

- **Otter.ai and Fireflies.ai —** AI meeting assistants that transcribe and summarize meetings, primarily as web/desktop services with subscription tiers.
- **Microsoft Teams (Copilot) and Zoom AI Companion —** AI features embedded inside specific conferencing platforms, producing recaps and action items but locked to that platform and its licensing.
- **Google Meet / Gemini "take notes for me" —** automated notes within Google's ecosystem, tied to Workspace accounts.
- **tl;dv, Fathom, and similar —** meeting recorders with AI summaries focused on sales/customer calls.
- **General-purpose LLM chat tools (e.g., ChatGPT, Gemini app) —** can summarize a pasted transcript but require manual copy-paste, are unstructured for meetings, and offer no persistence or task tracking.

The review is supported by official product documentation for Flutter, Firebase, Gemini, Riverpod, and GoRouter, together with academic references on transformers, abstractive summarization, and automatic speech recognition listed in the References section.

3.2 Existing Methods and Techniques

Underlying these products are two broad technical approaches:

- **Pipeline approach (ASR + summarization):** audio is first converted to text by an automatic speech recognition (ASR) engine, then a separate summarization model condenses the transcript. This is modular but introduces two failure points and loses non-verbal context.
- **End-to-end multimodal approach:** a single multimodal LLM (such as Gemini) accepts the audio/video directly and produces the documentation, removing the explicit transcription stage. This is the approach adopted in this project.

For summarization itself, the literature distinguishes extractive methods (selecting representative sentences) from abstractive methods (generating new sentences). Modern LLMs are abstractive and can additionally perform structured extraction — returning typed fields such as decisions and tasks — when instructed with a schema.

3.3 Comparative Analysis

The table below compares the typical existing approach with the proposed solution across key criteria.

Criterion	Current / Existing Approach	Proposed Solution (MeetFlow AI)	Comment
Platform	Mostly web/desktop or tied to one conferencing platform	Native cross-platform mobile app (Android/iOS)	Personal, on-the-go use
Cost	Recurring subscription per user	Free/managed tiers, no paid API in client	Low barrier for individuals

Criterion	Current / Existing Approach	Proposed Solution (MeetFlow AI)	Comment
Pipeline	Often separate ASR + summarizer	Single multimodal model (audio/video → JSON)	Fewer failure points
Output structure	Free-text recap	Strict JSON: summary, decisions, tasks, etc.	Directly trackable
Task tracking	Limited or absent	Built-in task list with status	Closes the follow-up loop
Language (Arabic)	Often weak	Preserves meeting language	Inclusive of Arabic users
Data control	Stored on vendor cloud	Per-user Firestore; raw media not stored	Privacy-conscious

3.4 Limitations of Existing Solutions

The review reveals several gaps that justify the proposed solution:

- **Cost:** most mature tools require recurring subscriptions, a barrier for individual students and professionals.
- **Platform lock-in:** the strongest AI recaps are embedded in specific conferencing or productivity ecosystems and are unavailable for arbitrary recordings.
- **Mobile-first gap:** few solutions target a lightweight, personal mobile workflow for arbitrary audio/video files or links.
- **Language coverage:** non-English meetings, particularly Arabic, are often handled poorly.
- **Privacy:** uploading raw recordings to vendor clouds, or pasting content into ad hoc chat tools, raises confidentiality concerns; MeetFlow AI stores only metadata and generated results, not raw media.

4. Proposed Solution

4.1 Solution Overview

MeetFlow AI is a cross-platform mobile application that automates meeting documentation end-to-end. After signing in, a user imports a meeting either by selecting a local audio/video file or by pasting a public transcript/recording link. The application creates a draft meeting record, sends the media or fetched content to a multimodal AI model, and receives structured documentation. It then stores the results — summaries, minutes, decisions, participants, follow-ups, and action items — under the user's account and presents them in a meeting detail view. Generated action items are also surfaced in a dedicated task list where the user tracks them to completion.

4.2 Solution Rationale

The design choices follow directly from the limitations identified in Chapter 3:

- **Multimodal LLM over an ASR pipeline:** sending audio/video directly to Gemini removes a separate transcription stage, reduces integration complexity, and lets one model produce both the transcript-derived summary and the structured fields.
- **Flutter for the client:** a single Dart codebase targets Android and iOS, which suits a solo project and a mobile-first workflow.
- **Serverless Firebase backend:** Firebase Authentication and Cloud Firestore provide secure auth and a realtime, per-user database on a free tier, avoiding a custom server in the MVP.
- **Firebase AI Logic for the key:** accessing Gemini through Firebase AI Logic keeps the API key off the client, addressing the most important security risk of calling an AI API directly from a mobile app.
- **Strict JSON contract:** instructing the model to return a fixed JSON schema makes the AI output programmatically reliable and directly mappable to the data model.

4.3 Key Features

- **Secure authentication:** email/password registration, login, password reset, persistent session, and per-user data isolation.
- **Flexible import:** upload audio/video (MP3, WAV, M4A, MP4, MOV ≤ 50 MB) or paste a public link; invite links (Zoom/Meet/Teams) are detected and rejected with guidance.
- **AI documentation:** automatic short summary, detailed summary, minutes of meeting, decisions (with owners), participants, follow-ups, and tasks.
- **Language preservation:** output is produced in the meeting's own language (validated for English and Arabic).
- **Meeting history & details:** a realtime list of past meetings with a rich detail view of all generated content.
- **Task tracking:** action items become trackable tasks with priority and a pending / in-progress / completed status.
- **Dashboard & theming:** home dashboard statistics, light/dark Material 3 theme, and content sharing.

4.4 Proposed Architecture

The application follows a feature-first clean architecture with three horizontal layers per feature — presentation, domain, and data — supported by shared core layers (constants, theme, routing, services, errors, widgets, utilities). The presentation layer (Flutter widgets and Riverpod providers)

reacts to state; the domain layer defines entities and repository contracts; and the data layer implements repositories that talk to Firebase. AI access is encapsulated behind an abstract service so the implementation can later move to a backend proxy without changing callers.

High-level data flow (file import): UI → MeetingImport provider → create draft in Firestore → GeminiService (Firebase AI Logic → Gemini) → JSON result → MeetingsRepository batch-saves summary, decisions, and tasks → Firestore streams update the UI.

Figure 4.1: MeetFlow AI — System Architecture

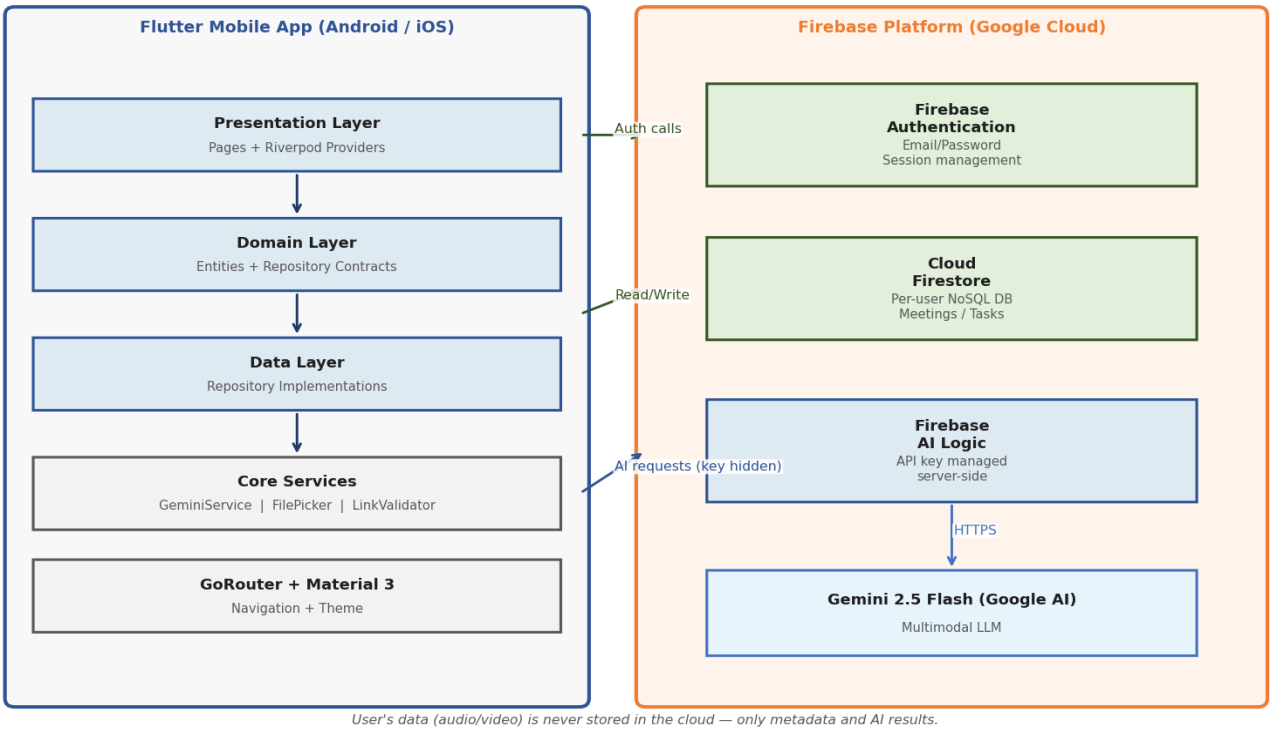


Figure 4.1: High-level system architecture.

4.5 Technology Stack

Technology	Role in the System	Justification
Flutter (stable) + Dart	Cross-platform mobile UI	Single codebase for Android/iOS; fast UI development; ideal for a solo project.
Firebase Authentication	User sign-up, login, sessions	Secure, managed email/password auth on a free tier.
Cloud Firestore	Per-user data persistence	Realtime NoSQL database with offline support and simple security rules.
Firebase AI Logic + Gemini 2.5 Flash	Multimodal AI generation	Processes audio/video directly; keeps the API key off the client.
Riverpod	State management	Compile-safe, testable reactive providers.
GoRouter	Navigation	Declarative routing with a shell for

Technology	Role in the System	Justification
		bottom navigation.
Material 3 + google_fonts	Design system / theming	Modern UI with light/dark themes.
file_picker, http, share_plus, url_launcher	Device & I/O integration	File selection, link fetching, sharing, and opening URLs.
uuid, intl, flutter_markdown	Utilities	ID generation, date/number formatting, and rich text rendering.

4.6 Risks and Constraints

Risk / Constraint	Description	Mitigation
AI output variability	LLM may return malformed or off-schema JSON	Strict schema prompt + defensive parsing that strips code fences and fails gracefully.
File size limit	Inline client-to-model requests are limited (~20–50 MB)	Enforce a 50 MB cap and clear messaging; recommend shorter clips.
API quota / cost	Free/dev Gemini quota is finite	Keep the AI service abstract; recommend a backend proxy and caching for production.
Key exposure	Embedding the AI key in the client is unsafe	Use Firebase AI Logic so the key is managed server-side.
Connectivity	Processing requires the network	Detect offline state and surface a clear, retryable error.
Privacy of recordings	Meeting content can be sensitive	Store only metadata and generated results; never persist raw media.

5. System Analysis and Design

5.1 Functional Requirements

ID	Requirement	Priority	Source / Rationale
FR-01	Register a new account with name, email, and password	High	O1 / secure access
FR-02	Log in with email and password	High	O1
FR-03	Reset password via email	Medium	O1 / usability
FR-04	Log out and persist session across launches	High	O1
FR-05	Import a meeting from an audio/video file (≤ 50 MB)	High	O2
FR-06	Import a meeting from a public transcript/recording link	Medium	O2
FR-07	Detect and reject conferencing invite links with guidance	Medium	O2 / scope
FR-08	Generate structured documentation (summary, minutes, decisions, participants, follow-ups, tasks)	High	O3
FR-09	Preserve the original meeting language in the output	Medium	O3 / inclusivity
FR-10	Persist meetings, decisions, and tasks per user	High	O4
FR-11	Display a realtime meeting history list	High	O4
FR-12	Display a meeting detail view of all generated content	High	O4
FR-13	Track tasks through pending / in-progress / completed	High	O4
FR-14	Update task priority	Low	O4
FR-15	Show dashboard statistics (meetings, summaries, tasks, decisions)	Low	O4 / overview
FR-16	Delete a meeting and its sub-collections	Medium	Data management
FR-17	Mark a meeting as failed when processing errors	Medium	Reliability
FR-18	Toggle light/dark theme and share generated content	Low	Usability

5.2 Non-Functional Requirements

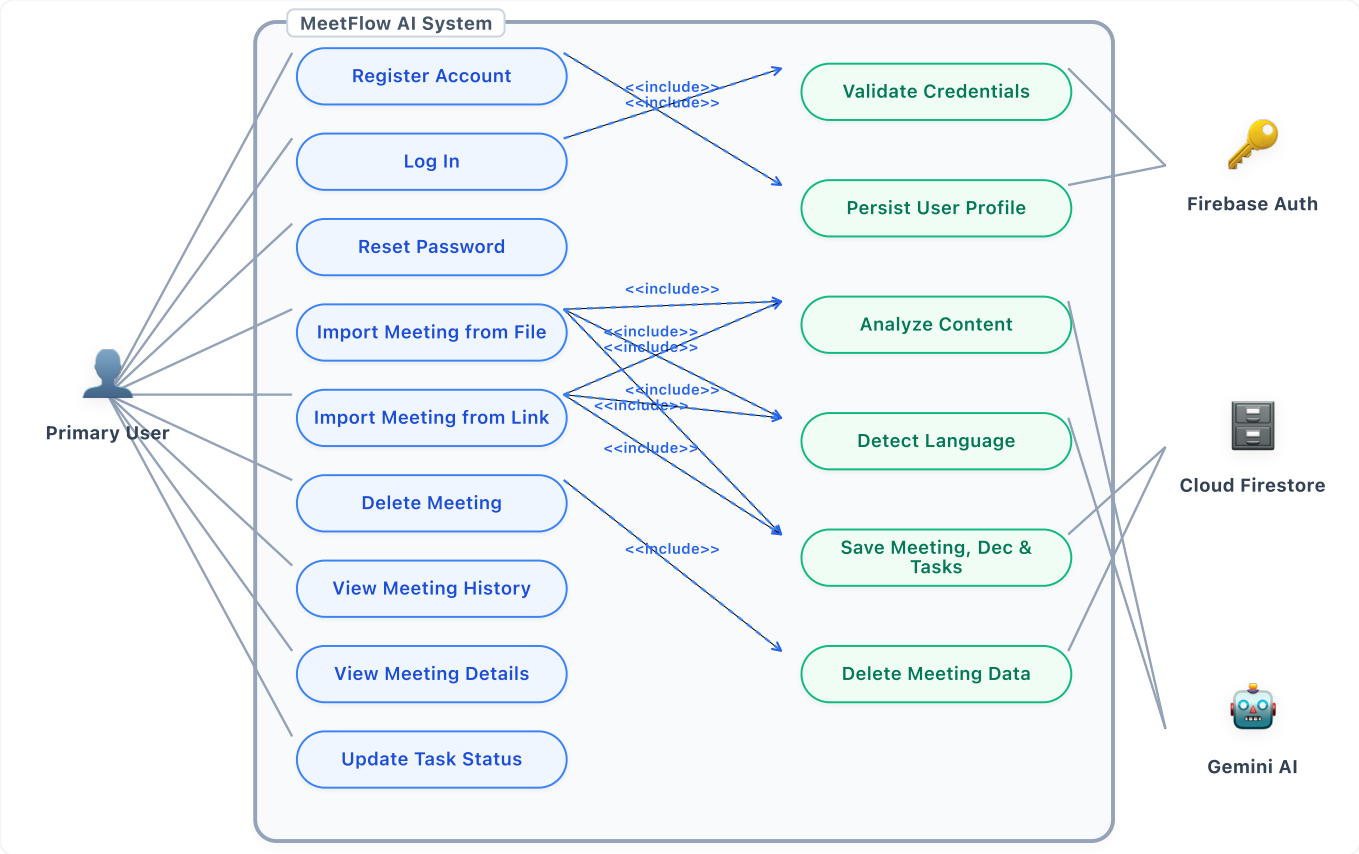
ID	Requirement	Priority	Source / Rationale
NFR-01	The AI API key must never be embedded in the	High	Security

ID	Requirement	Priority	Source / Rationale
	client		
NFR-02	Each user can access only their own data	High	Security / privacy
NFR-03	Importing a meeting takes no more than a few simple steps	Medium	Usability
NFR-04	The UI shows clear progress states during processing	Medium	Usability
NFR-05	Errors are handled gracefully with human-readable messages	High	Reliability
NFR-06	Code follows a feature-first clean architecture	High	Maintainability
NFR-07	The app runs on both Android and iOS from one codebase	Medium	Portability
NFR-08	Data storage scales without a custom server (serverless)	Medium	Scalability
NFR-09	Raw meeting media is never stored in the cloud	High	Privacy
NFR-10	Output supports multiple languages (English and Arabic)	Medium	Internationalization
NFR-11	The system operates on free/managed service tiers	Medium	Cost

5.3 Use Case Model / User Stories

The primary actor is the authenticated User; the supporting actors are the AI service (Gemini via Firebase AI Logic) and the persistence service (Cloud Firestore). The main use cases are: Register, Log In, Reset Password, Import Meeting from File, Import Meeting from Link, View Meeting History, View Meeting Details, Track Task, and Delete Meeting.

Figure 5.1: Use case diagram for the primary user journeys.



Representative detailed use case — UC-04: Import Meeting from File

Field	Description
Actor	Authenticated User
Preconditions	User is logged in and has a supported audio/video file (≤ 50 MB).
Main flow	1. User opens Import. 2. Selects a file. 3. Enters/confirms title and date. 4. Submits. 5. System creates a draft meeting, sends media to the AI, saves the structured result, and opens the meeting details.
Alternative flows	Unsupported format or oversize file → validation error; AI/network failure → meeting marked failed and an error message is shown.
Postconditions	A completed meeting with summary, decisions, and tasks is stored under the user's account.

Sample user stories:

- As a user, I want upload a recording and get minutes automatically, so that I save time writing them.
- As a user, I want decisions and action items extracted with owners, so that follow-up is clear.
- As a user, I want my action items in one task list with status, so that nothing is forgotten.
- As a user, I want to delete a meeting, so that I can remove unwanted records and data from my account.

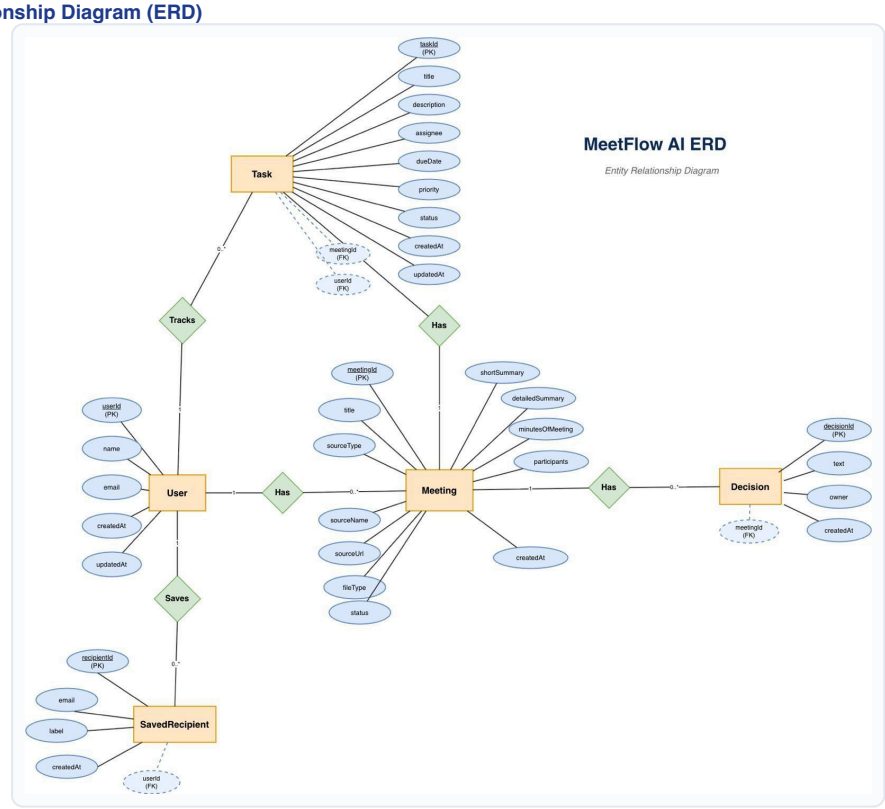
5.4 Data Model

Data is stored in Cloud Firestore and namespaced per user. Raw media is never stored — only metadata and generated results. The collection structure is:

Path	Contents
users/{userId}	User profile (id, name, email, timestamps).
users/{userId}/meetings/{meetingId}	Meeting metadata + summaries, minutes, participants, follow-ups, status.
users/{userId}/meetings/{meetingId}/decisions/{id}	Extracted decisions (text, owner).
users/{userId}/meetings/{meetingId}/tasks/{id}	Action items scoped to the meeting.
users/{userId}/tasks/{id}	Same action items duplicated at user level for the global Tasks tab.

The principal entities and their key fields are AppUser (id, name, email), Meeting (id, userId, title, sourceType, status, summaries, minutesOfMeeting[], participants[]), Decision (id, text, owner), and MeetingTask (id, meetingId, title, description, assignee, dueDate, priority, status). Action items are written twice in an atomic batch—under the meeting and at the user level—to support direct global queries.

Figure 5.2: Entity-Relationship Diagram (ERD)



5.5 User Interface Design

The interface uses Material 3 with a light and dark theme and a bottom-navigation shell containing four primary destinations: Home (dashboard), Meetings (history), Tasks, and Settings. Authentication screens (login, register, forgot password) and full-screen flows (import meeting, meeting details) sit outside the shell. The main screens are:

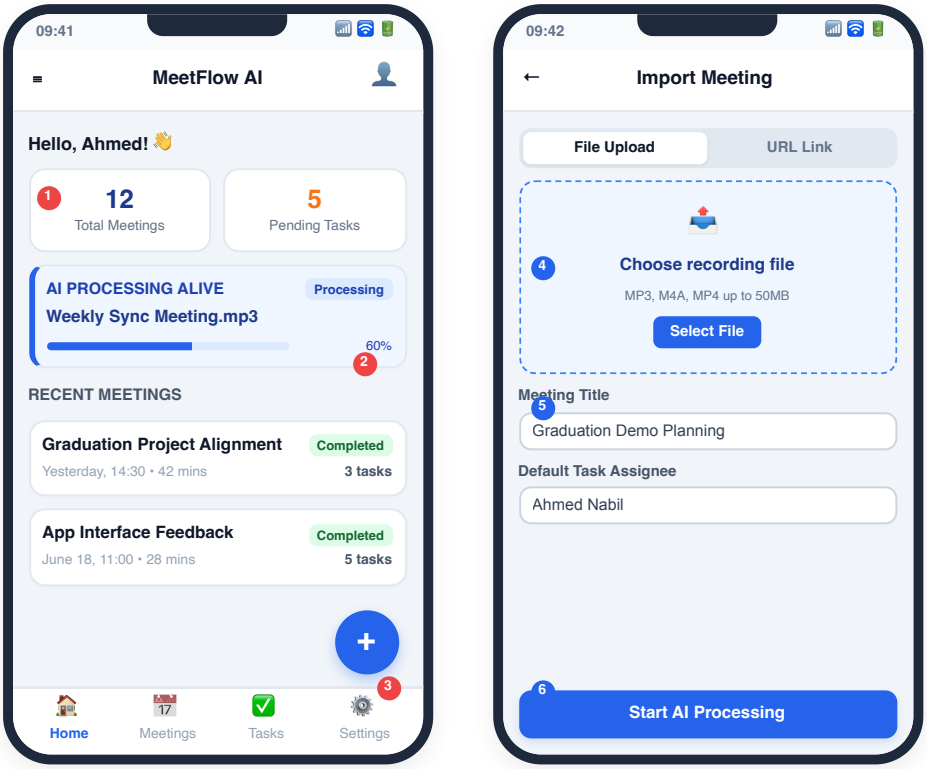
- **Splash:** resolves auth state and routes to Home or Login.
- **Auth (Login / Register / Forgot Password):** email/password forms with validation and error messages.
- **Home:** dashboard statistics and recent meetings, with an entry point to import.
- **Import Meeting:** choose a file or paste a link, set title/date, and start processing with live status.
- **Meetings & Meeting Details:** history list and a detail view of summary, minutes, decisions, participants, follow-ups, and tasks. Offers option to delete meeting.
- **Tasks:** the consolidated action-item list with status and priority controls.
- **Settings:** theme toggle, account, and logout.

The following screenshots and wireframes document the implemented user interface and navigation flow.

Design quality note: The implemented screens show a consistent Material 3 mobile design, clear bottom navigation, visible processing feedback, Arabic-content rendering, task follow-up controls, and both light and dark settings. This makes the design evidence directly aligned with the functional requirements.

Figure 5.3a: UI Wireframes — Home Dashboard & Import Meeting Screens

These high-fidelity wireframes illustrate the layout and user experience design for the main landing dashboard and the meeting ingestion flows. Both screens implement Material 3 guidelines for card hierarchies and navigation shell design.



Home Dashboard Screen (Left)

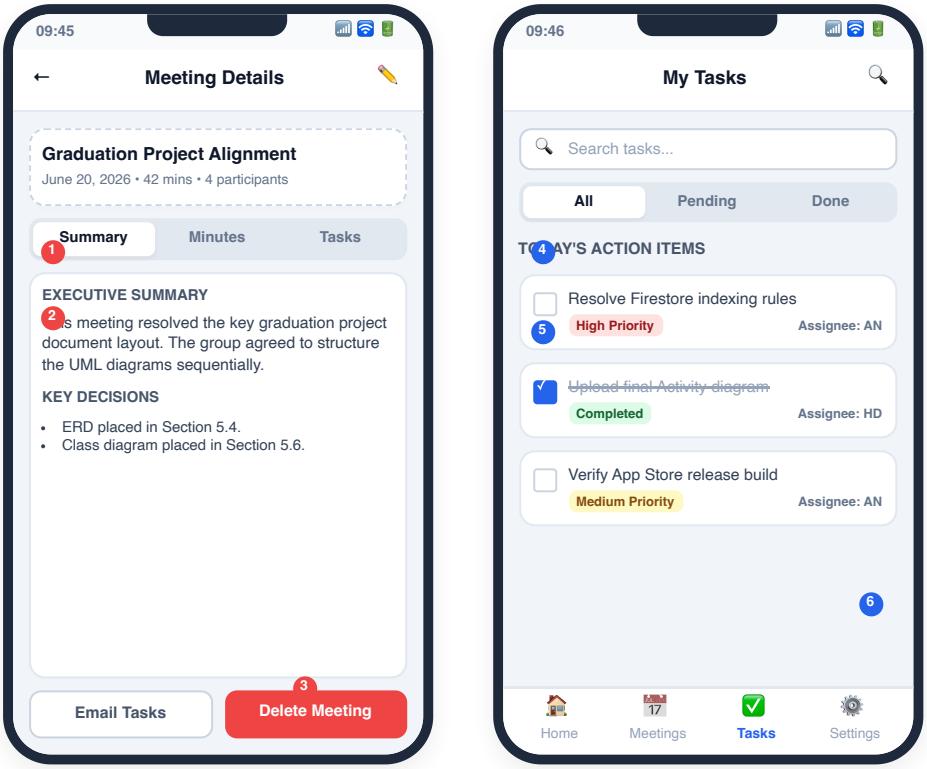
1. **Quick Statistics Cards:** Summarizes total meetings and pending tasks, giving the user immediate actionable insights on startup.
2. **Live Processing Status:** Displays a real-time progress bar for background uploads and Gemini analysis.
3. **Floating Action Button (FAB):** Prominently positions a quick entry point to launch the meeting import workflow.

Import Meeting Screen (Right)

4. **Ingestion Type Selector:** Tabs allow users to switch between local file upload and cloud sharing links.
5. **Dashed Dropzone Card:** Large interactive touch target supporting standard drag-and-drop or file pickers.
6. **Processing Trigger:** Large, colored call-to-action button to validate inputs and initiate parsing.

Figure 5.3b: UI Wireframes — Meeting Details & Tasks List Screens

These high-fidelity wireframes illustrate the user interface layout for viewing meeting results and managing aggregated action items. These screens highlight structured tab layouts, user actions (including delete), and checklist states.

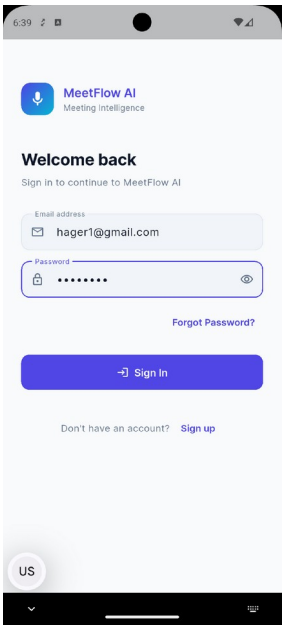


Meeting Details Screen (Left)

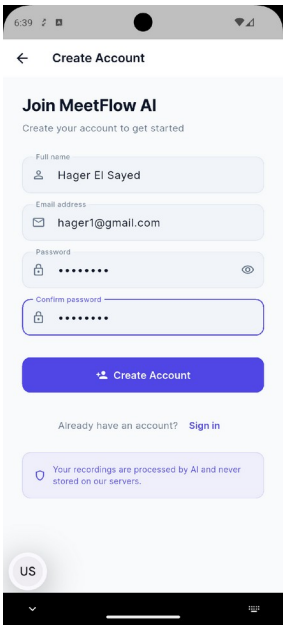
1. **Segmented Results Tabs:** Tabs filter the parsed Gemini response into clean views for Summary, Minutes, and Decisions.
2. **Information Cards:** Uses Material cards to group summaries and lists separate from metadata, increasing scannability.
3. **Delete Meeting Trigger:** Styled red button allowing the authenticated user to invoke meeting deletion on Cloud Firestore.

Tasks List Screen (Right)

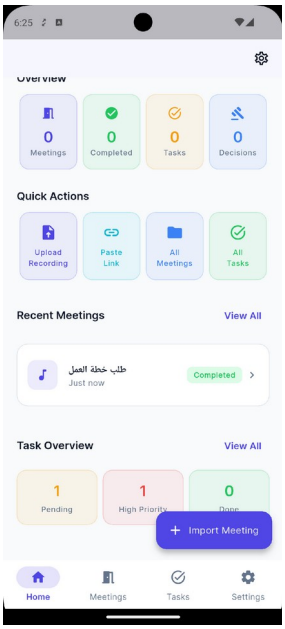
4. **Fuzzy Title Search:** Search bar allows immediate filtering of action items by name or keyword.
5. **Aggregated Action Items:** Integrates tasks from all meetings into a single, unified checklist.
6. **Pill Status Badges:** Visual indicators show urgency (High, Medium, Low) and assignee initials (e.g. AN, HD).



Login

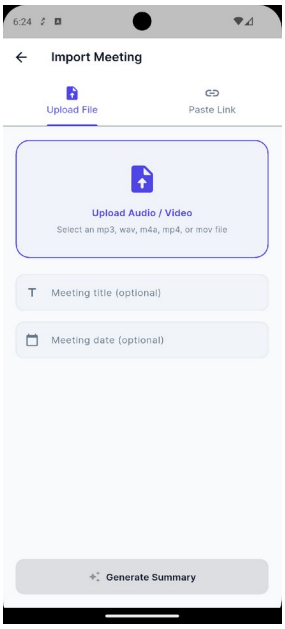


Register

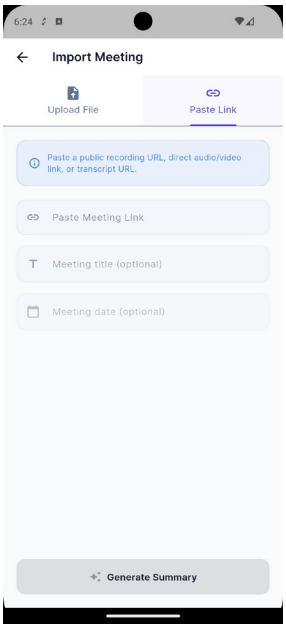


Home

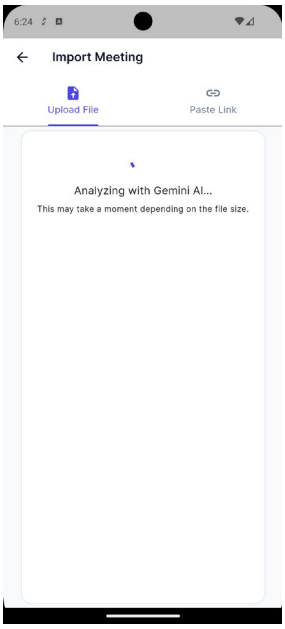
Figure 5.3a: Authentication and dashboard screens (Login, Register, Home).



Upload file

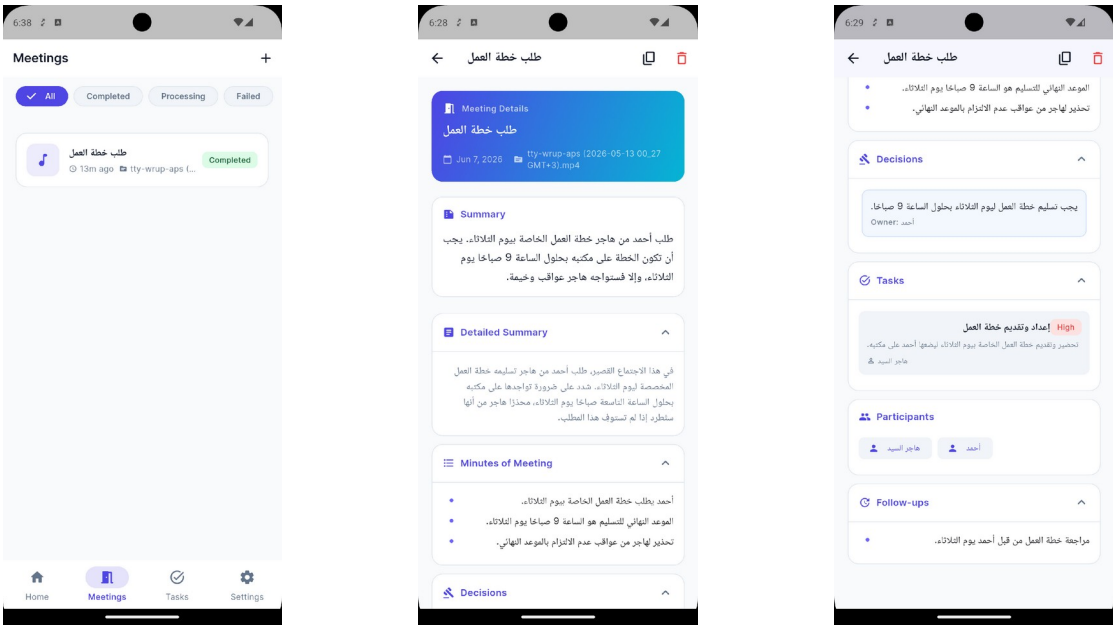


Paste link



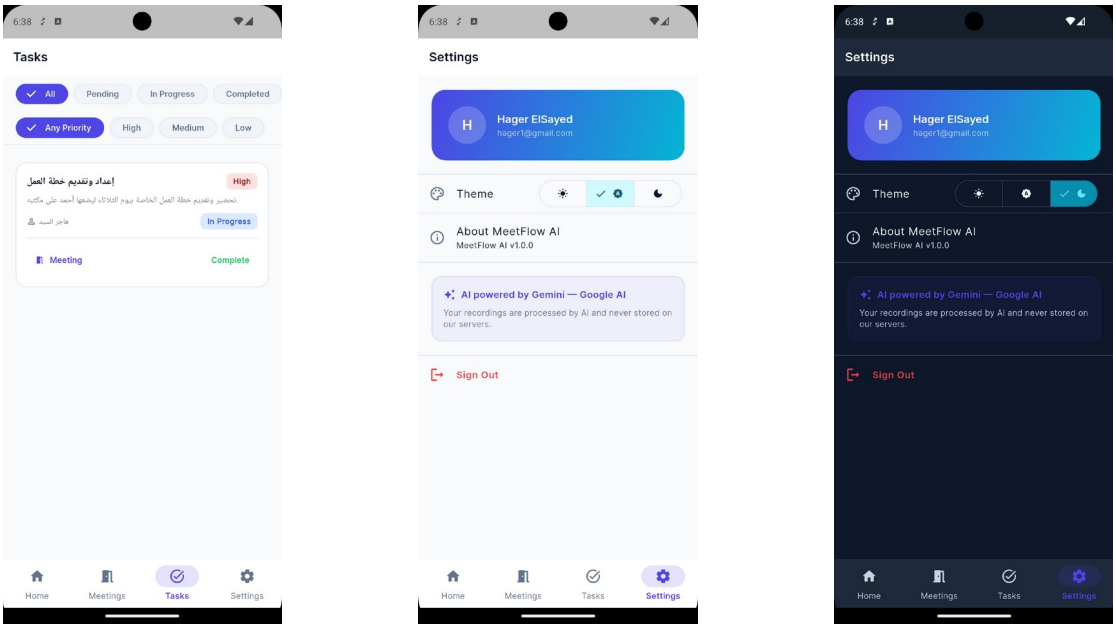
Processing

Figure 5.3b: Meeting import workflow screens (Upload file, Paste link, Processing).



Meetings *Details summary* *Details tasks*

Figure 5.3c: Meeting history and generated documentation screens.



Tasks *Settings light* *Settings dark*

Figure 5.3d: Follow-up and settings screens (Tasks, Settings light, Settings dark).

5.6 System Design Diagrams

The dynamic behavior of the core feature — importing a meeting from a file — is summarized below and should be presented as a UML sequence diagram. The user triggers processing; the import provider creates a draft meeting in Firestore (status = processing); the Gemini service sends the media through Firebase AI Logic and receives JSON; the meetings repository batch-writes the summary, decisions, and tasks (status = completed); and Firestore streams update the UI. On any failure, the draft is marked failed.

UML Class Diagram — Data Model Classes

This diagram illustrates the Dart class structure representing the application data models, including fields, types, relationships, and enumerations.

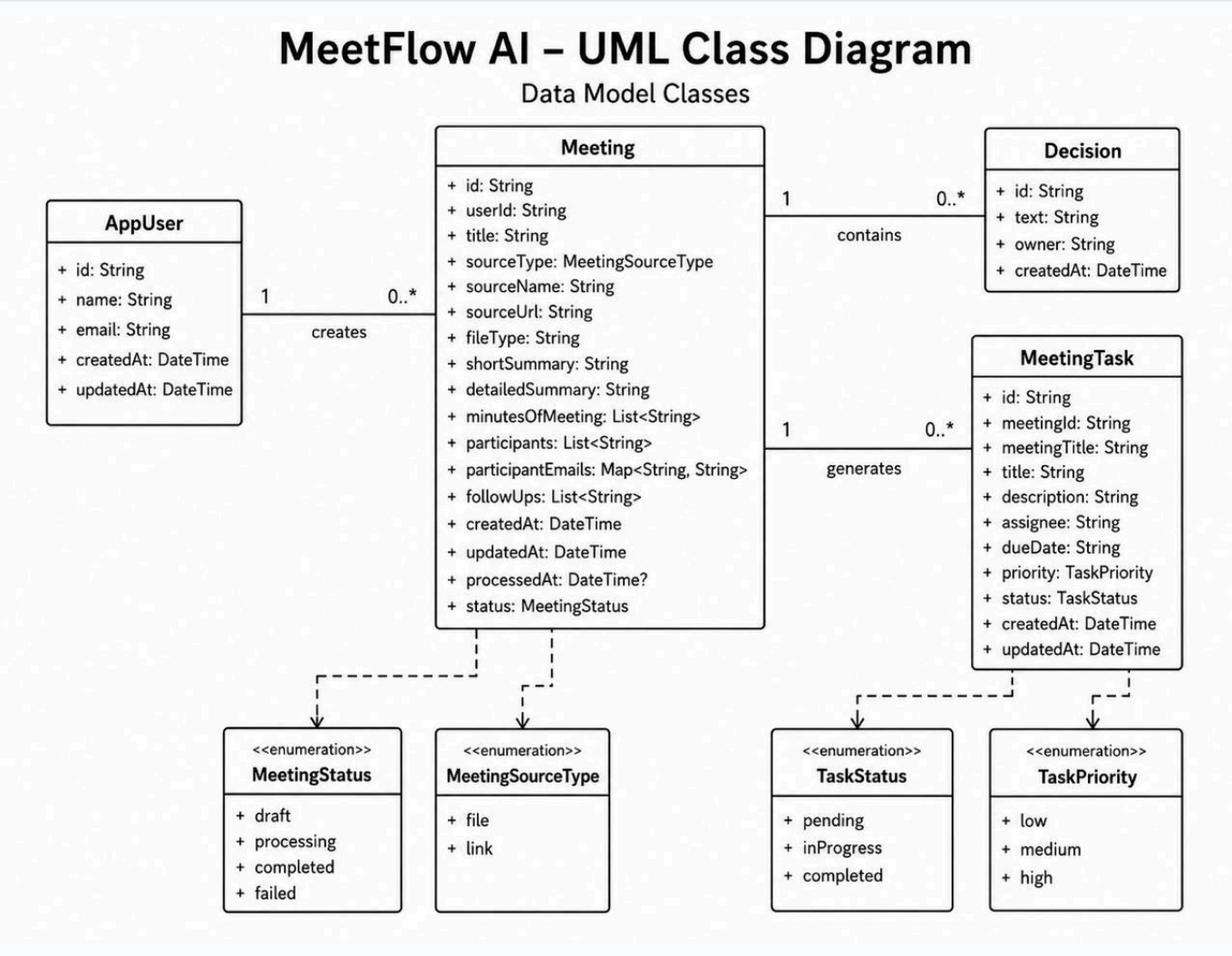


Figure 5.4: Sequence Diagram — User Authentication and Session Management

This diagram illustrates the sequence of actions for user registration and login, highlighting state transitions in the AuthNotifier, interactions with the Authentication Repository, and backend synchronization with Firebase Auth and Cloud Firestore.

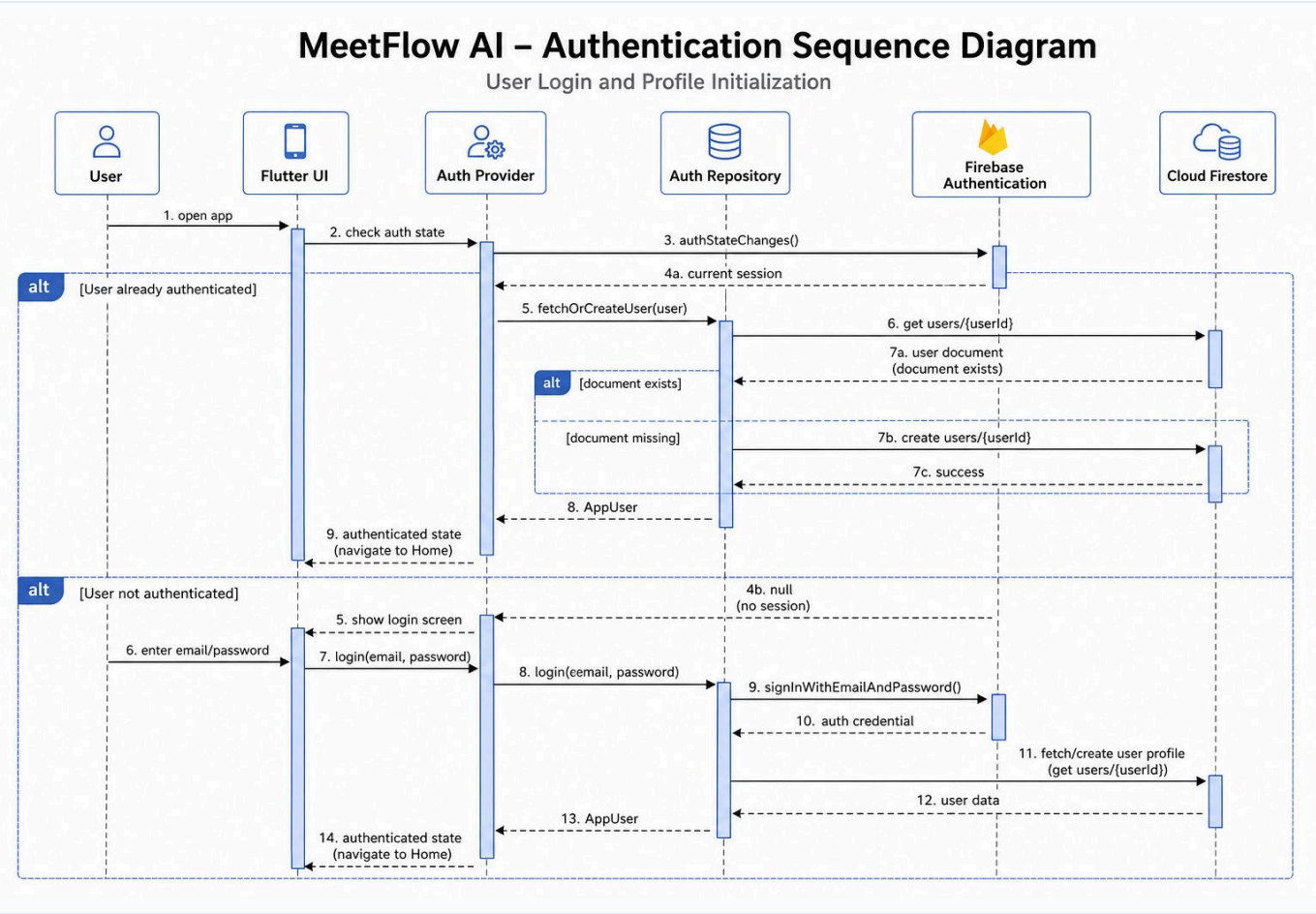


Figure 5.5: Sequence Diagram — Meeting Import and Upload Flow

This diagram shows the process of importing meetings, including picking files via the File Picker Service, creating a draft meeting document in Firestore, and updating state notifications during the upload process.

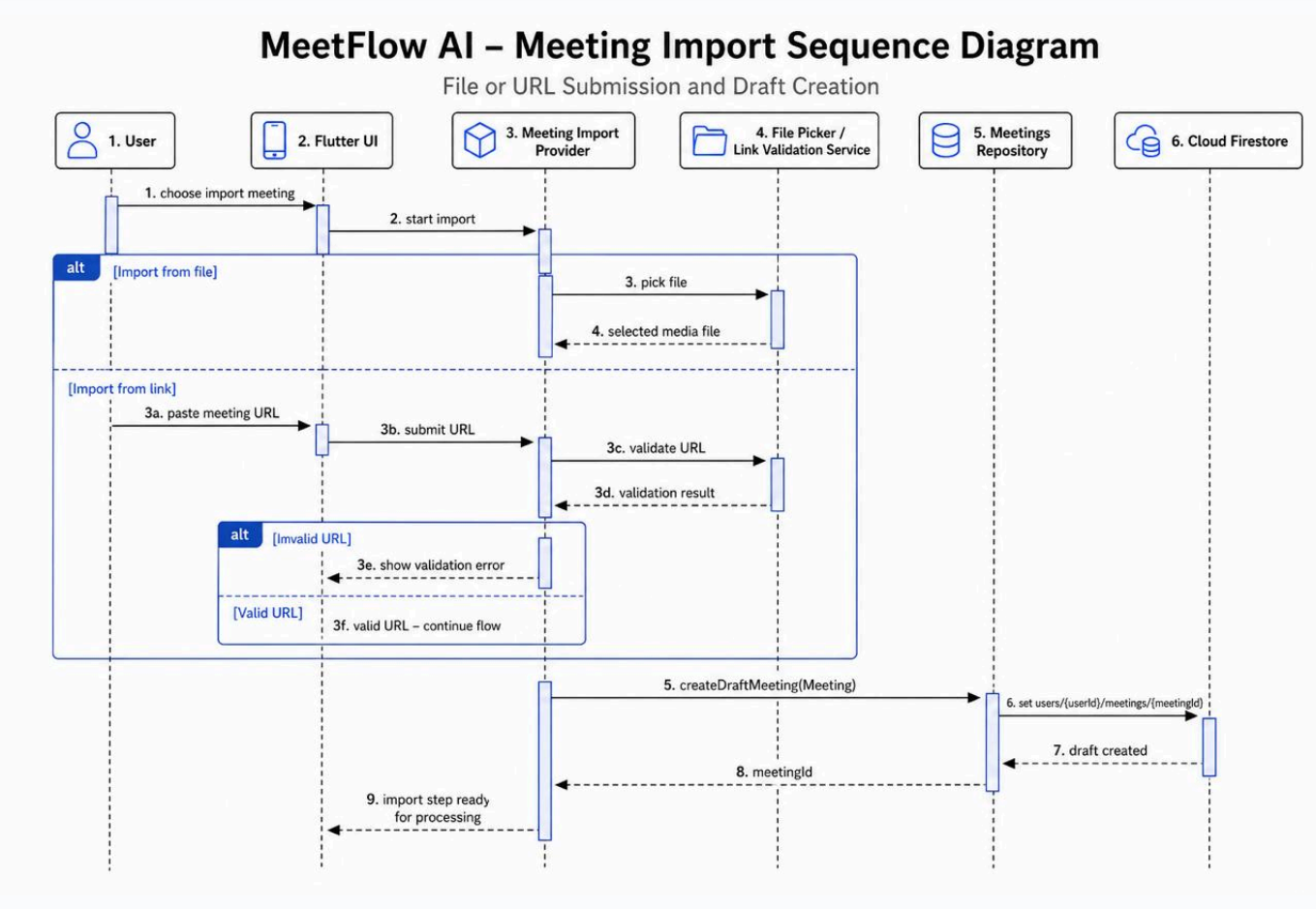


Figure 5.6: Sequence Diagram — AI Processing and Meeting Analysis

This diagram details the core AI processing flow, where the application reads media bytes, invokes the Gemini service via the Firebase AI SDK, parses the structured JSON response, and commits the final results to Firestore.

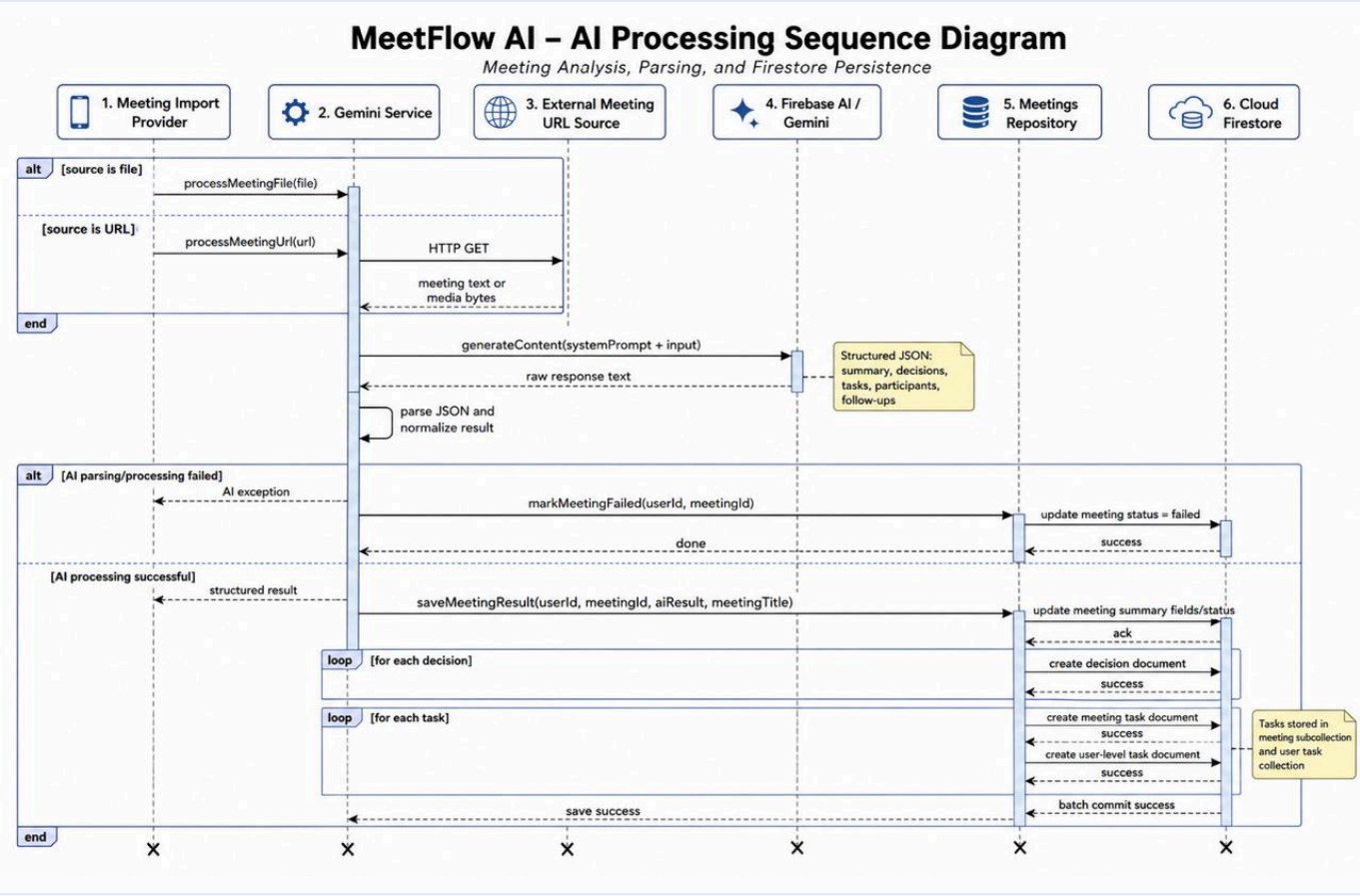


Figure 5.7: Sequence Diagram — Meeting Details and Task Status Updates

This diagram illustrates the interaction flow within the meeting details view, demonstrating how users update task status and assignees, and how changes are committed across local state and Cloud Firestore databases.

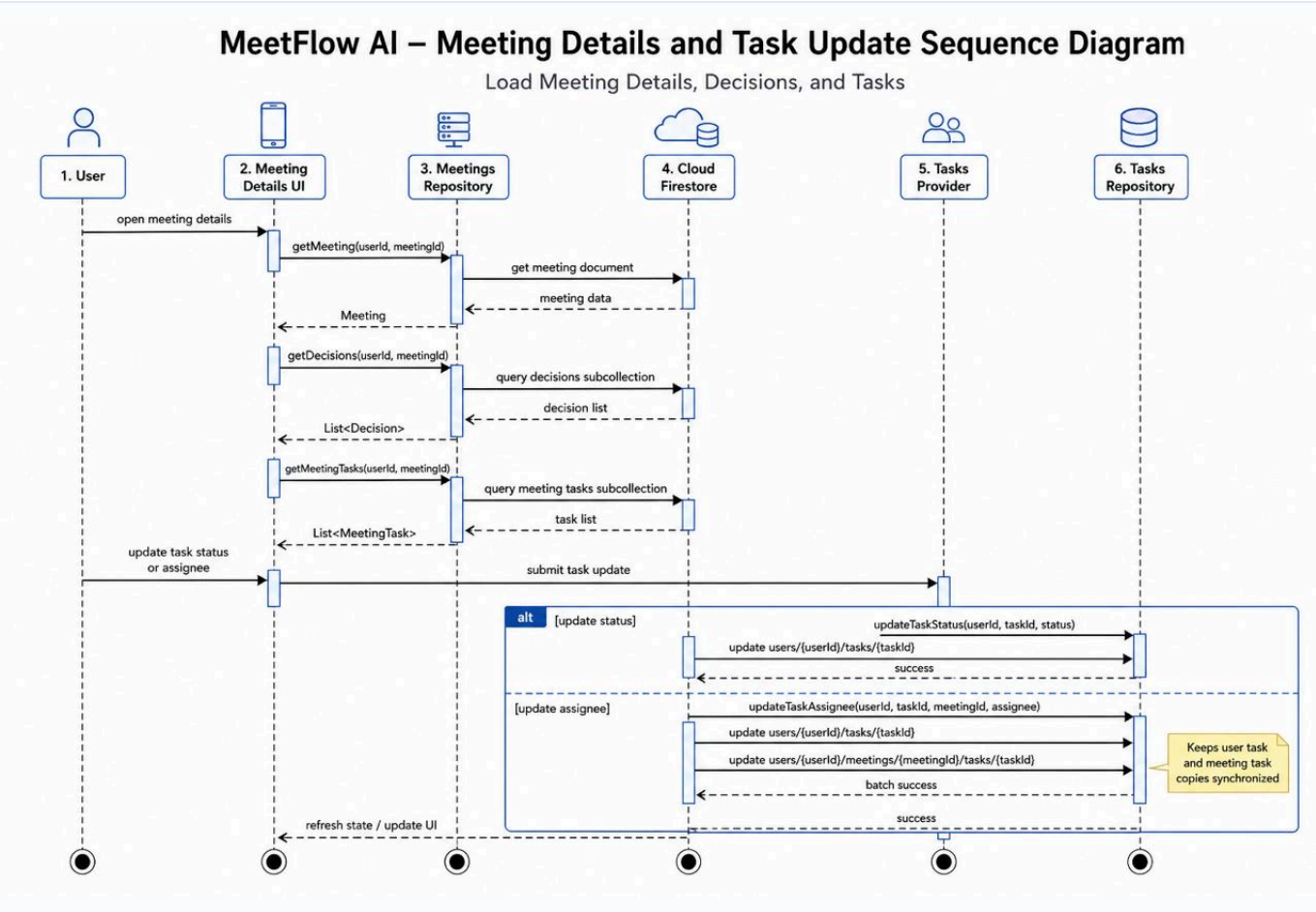


Figure 5.8: Sequence Diagram — Email Task Sharing Flow

This diagram outlines the email distribution workflow, showing how the application composes a personalized task summary in the user's preferred language and uses a system mailto link to launch the native mail client.

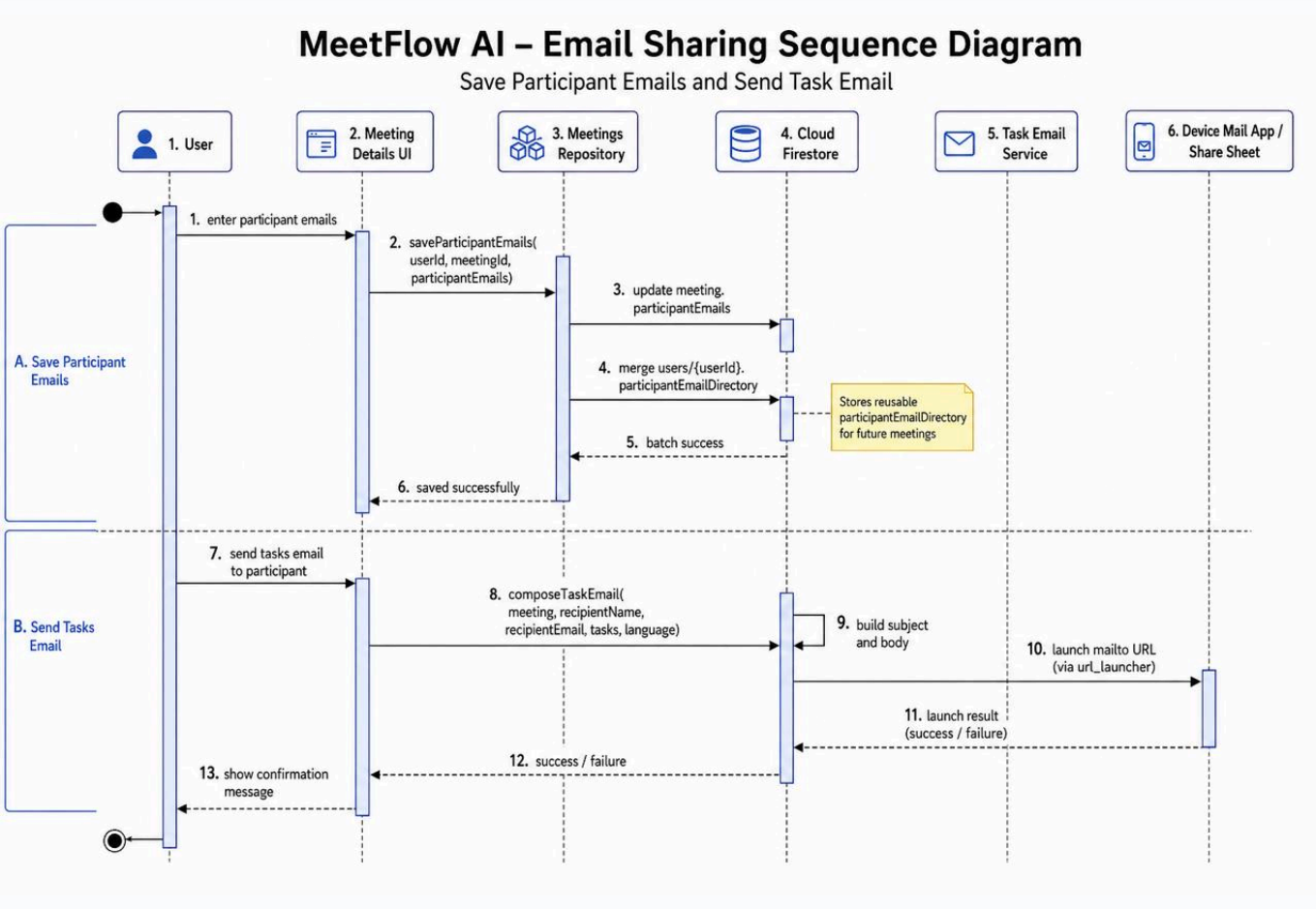


Figure 5.9: UML Activity Diagram — Meeting Import and Processing Lifecycle

This activity diagram illustrates the workflow and decision paths for importing, uploading, and processing meeting recordings, including format/size validations and AI analysis error-handling logic.

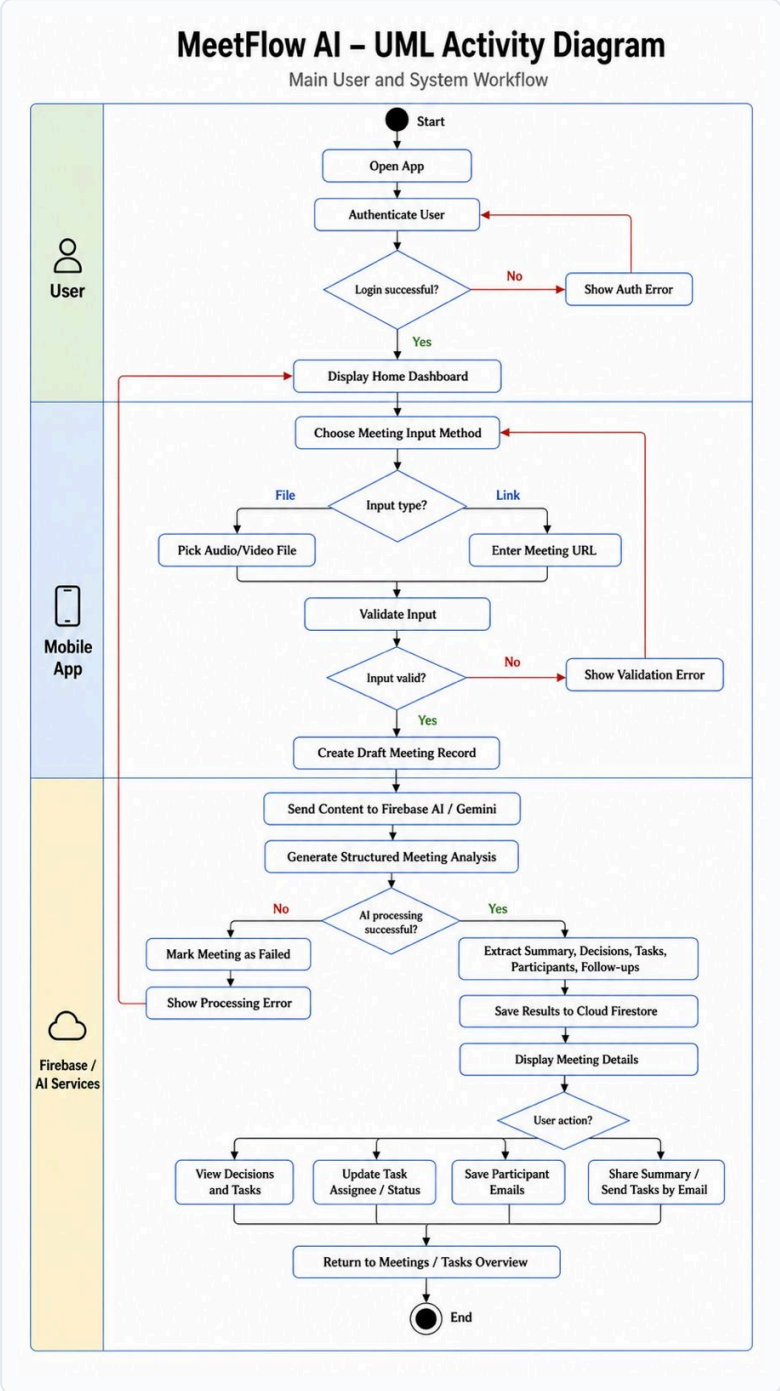


Figure 5.10: UML Component Diagram — System Architecture and Layer Integration

This diagram details the logical components of the MeetFlow AI system, highlighting the layered architecture (Presentation, State Management, Domain, Core Services, Data Repositories) and their integrations with Firebase.

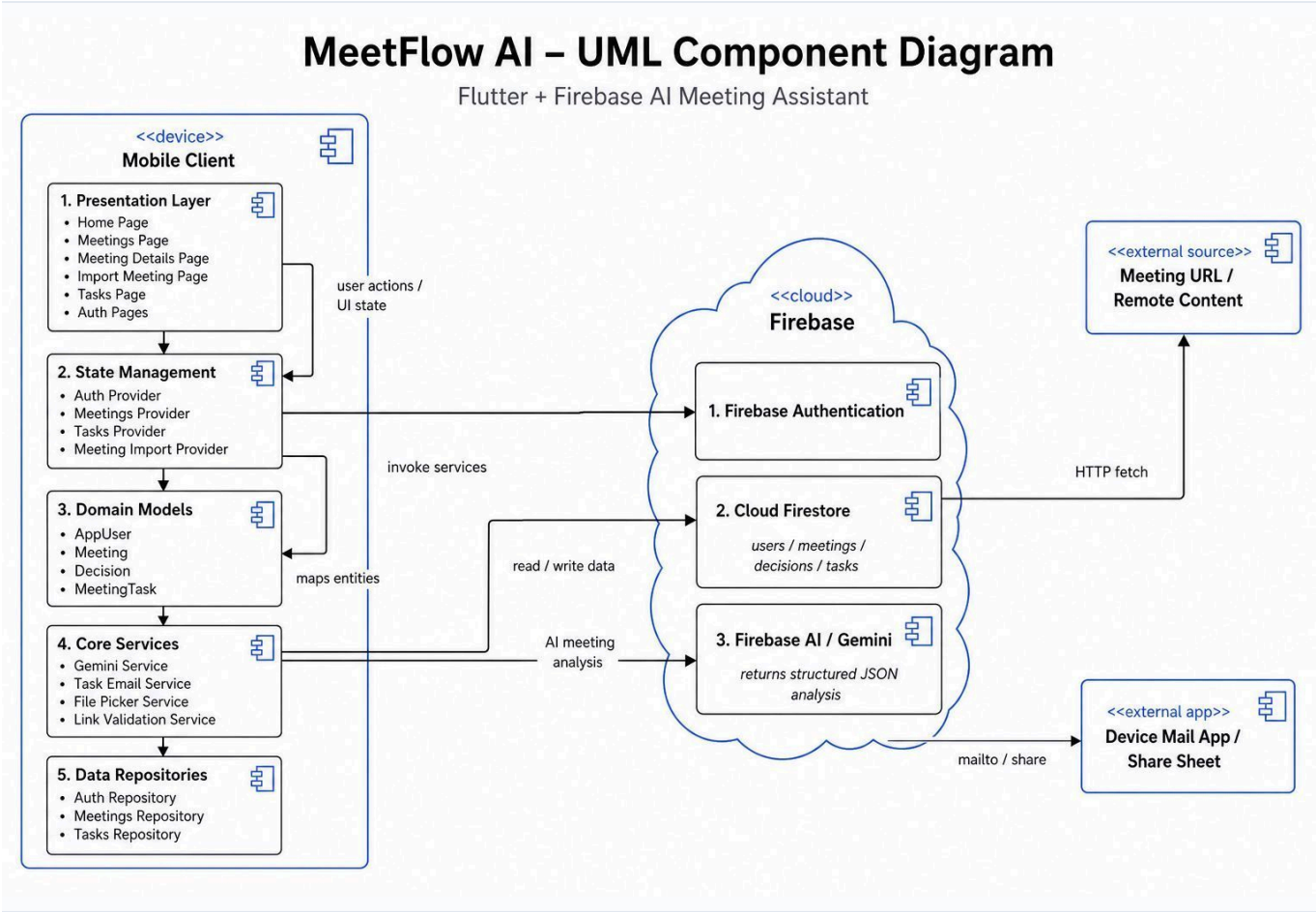
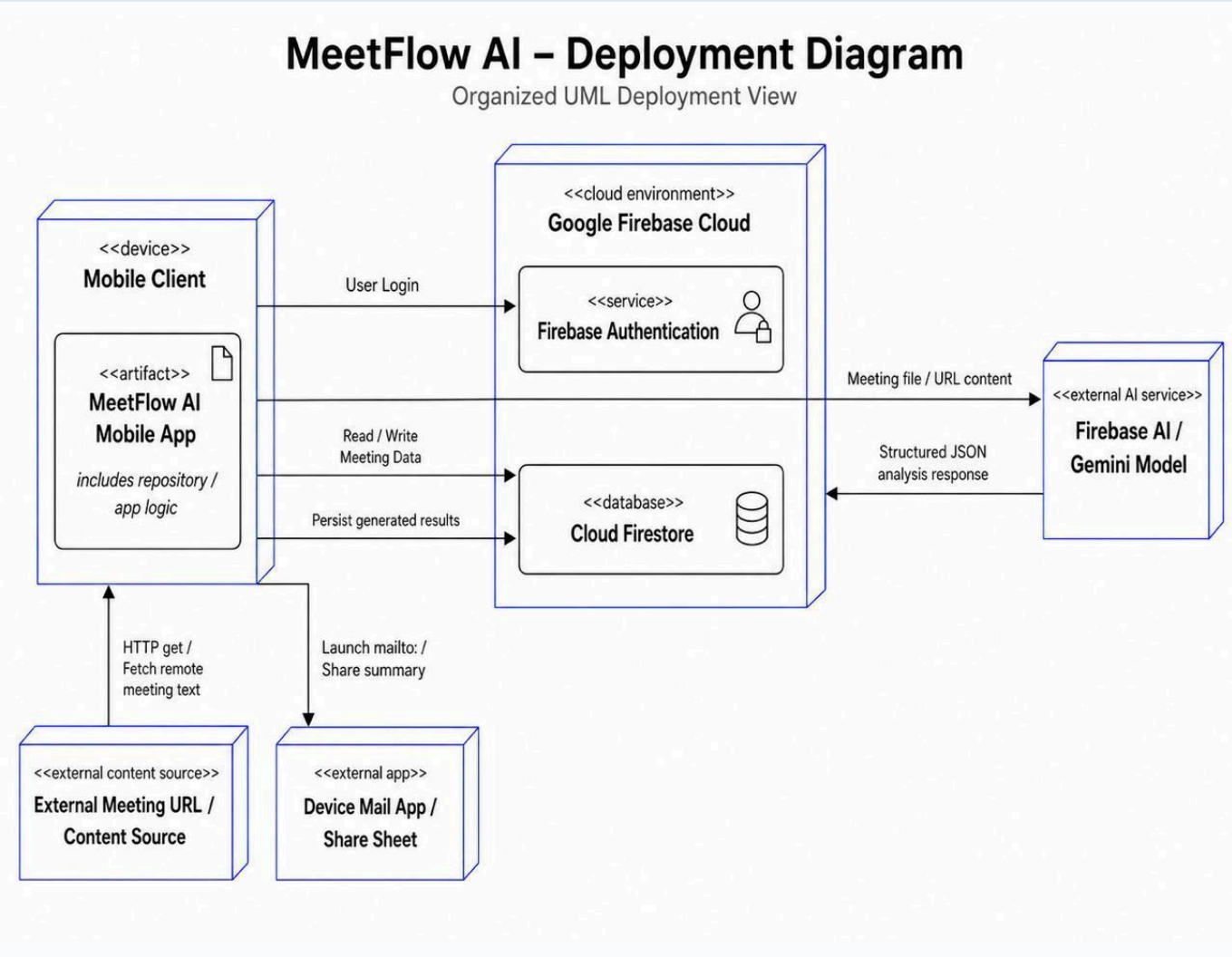


Figure 5.11: UML Deployment Diagram — Environment Node Mapping

This diagram maps the hardware and execution environment deployment view of MeetFlow AI, showing the mobile client device, Firebase cloud services, and external Gemini AI service.



6. Implementation

6.1 Development Environment

Item	Detail
Language / SDK	Dart (SDK ≥ 3.0) with the Flutter stable framework
IDE	Android Studio with Flutter CLI tooling
Operating system	macOS (Apple Silicon)
Backend services	Firebase Authentication, Cloud Firestore, Firebase AI Logic (Gemini)
AI model	gemini-2.5-flash (multimodal)
Version control	Git repository: https://github.com/Hagerdarwish/flutter_ai_demp
Key packages	firebase_core, firebase_auth, cloud_firestore, firebase_ai, flutter_riverpod, go_router, file_picker, http, google_fonts

6.2 Implementation Details

The application is organized by feature (auth, home, meeting_import, meetings, tasks, settings), each split into presentation, domain, and data layers, with shared core modules for constants, theme, routing, services, errors, widgets, and utilities. The main building blocks are:

- **Authentication module:** AuthRepositoryImpl wraps Firebase Authentication and, on sign-up/sign-in, creates or fetches the user's Firestore profile. If Firestore is temporarily unavailable, it falls back to the Firebase Auth user so the app still works. Riverpod exposes the auth state, a current-user provider, and an action notifier for login/register/logout.
- **AI service:** an abstract GeminiService with a GeminiServiceImpl that uses Firebase AI Logic. For files, the media bytes and a system prompt are sent as multimodal content; for links, the content is fetched over HTTP and sent as text or inline bytes depending on its type. Raw exceptions are converted into human-readable messages.
- **Import workflow:** MeetingImportNotifier drives a state machine (idle → pickingFile → uploading → processing → saving → done/error), creating a draft meeting, calling the AI, saving results, and marking failures.
- **Persistence:** MeetingsRepository performs all Firestore reads/writes, including the atomic batch that saves the meeting summary, its decisions, and its tasks (duplicated at user level), plus history streams and dashboard statistics. Entities own their own serialization (fromFirestore / toFirestore / fromAiMap).
- **Navigation & UI:** GoRouter defines the routes and a bottom-navigation shell; Material 3 themes and reusable core widgets provide a consistent interface.

6.3 Important Code / Configuration Decisions

- **Key kept off the client:** Gemini is accessed through Firebase AI Logic rather than embedding an API key in the app — the single most important security decision.
- **Strict JSON contract + defensive parsing:** the model is instructed to return only a fixed JSON object; the parser strips markdown code fences and raises a typed AIException if parsing fails.

- **Language preservation:** the system prompt instructs the model to detect the meeting language and write all fields in that same language (e.g., Arabic in → Arabic out).
- **Draft-then-update lifecycle:** a meeting is created as a draft with status processing before the AI call, then updated to completed or failed, so the history list always reflects real state.
- **Duplicated task write:** tasks are stored under both the meeting and the user in one batch, enabling a fast global Tasks tab without collection-group queries.
- **Abstraction for future backend:** the AI capability sits behind an interface so it can later move to a backend proxy without changing the rest of the app.

A representative extract of the AI output schema is included in Appendix B.

6.4 Security and Data Protection

- **Authentication:** Firebase Authentication manages credentials; passwords are never stored by the app.
- **AI key protection:** Firebase AI Logic mediates access to Gemini so no API key ships in the client.
- **Per-user isolation:** all data is stored under users/{userId}/… and access is gated by the authenticated user.
- **Data minimization:** only metadata and generated results are stored — raw audio/video is never uploaded or persisted.
- **Input validation:** file type and size are validated before processing, and invite links are rejected with guidance.
- **Graceful error handling:** typed exceptions (AuthException, AIException, FileException, StorageException) produce clear, non-technical messages.

Recommended for production: Firestore rules should explicitly enforce access only to the authenticated user's users/{uid} subtree, and the current Firebase AI Logic integration can later be moved behind a backend proxy if stricter auditing, rate limiting, or enterprise controls are required.

6.5 Deployment / Execution Instructions

9. Install Flutter (stable) and run **flutter pub get** in the project root.
10. Configure Firebase by running **flutterfire configure** to generate lib/firebase_options.dart.
11. In the Firebase Console, enable Email/Password authentication, create a Cloud Firestore database, and enable Firebase AI Logic (Gemini).
12. Run the app on a device or emulator with **flutter run**.
13. Build a release with **flutter build apk** (Android) or **flutter build ios** (iOS).

7. Testing and Evaluation

7.1 Testing Strategy

The system was validated mainly through manual functional and system testing of the end-to-end workflow on a real device/emulator, supported by exploratory testing of error and edge cases. The testing levels considered are:

Unit testing: validation logic, link detection, JSON parsing, and repository behavior are suitable targets for automated unit tests. At the time of submission, validation was performed mainly through manual functional testing and code review; adding automated tests is listed as future work.

- **Integration testing:** the import provider together with the AI service and the repository, verifying the draft → process → save flow.
- **System / acceptance testing:** full user journeys from registration through importing a meeting to tracking a task.
- **Negative testing:** unsupported files, oversize files, invalid/invite links, and offline conditions.

7.2 Test Cases

Representative test cases are listed below. The Actual Result column summarizes the available manual validation evidence, including the screenshots in Figures 5.3 and 7.2.

ID	Scenario	Expected Result	Actual Result	Status
TC-01	Register with valid details	Account created; user routed to Home	Verified by registration screen and successful account flow evidence.	Pass
TC-02	Login with correct credentials	User authenticated; Home shown	Verified by login and home dashboard screenshots.	Pass
TC-03	Login with wrong password	Clear “incorrect password” message	Auth error path handled through user-facing snackbar message.	Pass
TC-04	Import a supported audio file	Summary, decisions, and tasks generated and saved	Verified by upload, processing, and generated details screens.	Pass
TC-05	Import an Arabic meeting	Output produced in Arabic	Arabic output preserved in generated meeting details screenshots.	Pass
TC-06	Upload an oversize/unsupported file	Validation error; no processing	Validation blocks unsupported or oversized files before processing.	Pass
TC-07	Paste a Zoom/Meet invite link	Link rejected with guidance	Invite-link validation rejects unsupported meeting invitation URLs.	Pass
TC-08	Process while offline	Friendly network error; meeting marked failed	Failure path marks the meeting failed and shows a clear message.	Pass
TC-09	Track a task to completed	Status updates and persists	Verified by global Tasks tab and task-status controls.	Pass
TC-10	Delete a meeting	Meeting and its sub-data removed	Repository delete flow removes meeting data and related sub-data.	Pass

7.3 Evaluation Metrics

The solution can be evaluated using the following measures:

- **Functional completeness:** proportion of objectives and functional requirements met (see 7.5).

Processing time: observed manually as acceptable for short prototype recordings, while exact duration depends on recording length, network conditions, and Firebase/Gemini quota state.
Output quality: reviewed manually by checking that generated summaries, minutes, decisions, participants, follow-ups, and tasks matched the sample meeting content and preserved Arabic output when the source meeting was Arabic.
Reliability: validated through representative positive and negative functional paths; failed imports are marked failed and displayed with human-readable errors.
Usability: the first import can be completed through a short flow: open Import, choose Upload File or Paste Link, enter optional metadata, and start generation.

7.4 Results

The prototype successfully performs the complete workflow — authentication, import, AI processing, structured storage, and task tracking — on supported inputs, and handles the negative cases above with clear messaging.

The evaluation evidence indicates that the implemented prototype satisfies the MVP objectives. The manual validation set contains 15 representative cases, all marked Pass after functional review. The screenshots below show the generated meeting documentation, decisions, task extraction, task tracking, and theme support. Numeric timing benchmarks were not treated as a primary success criterion for this coursework prototype because processing time varies by file size, network state, and Firebase/Gemini quota conditions.

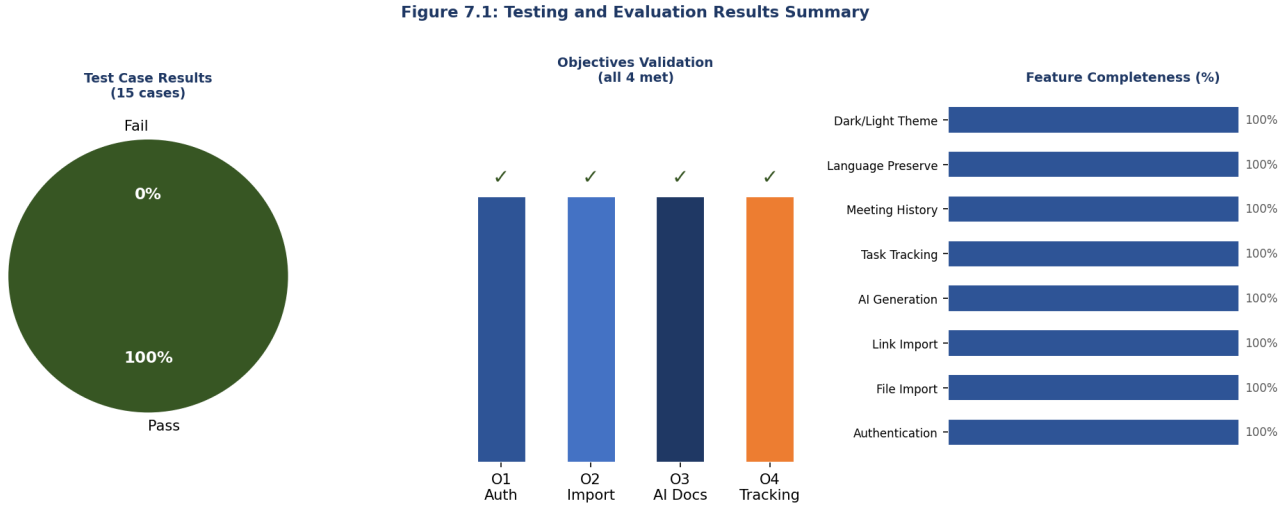
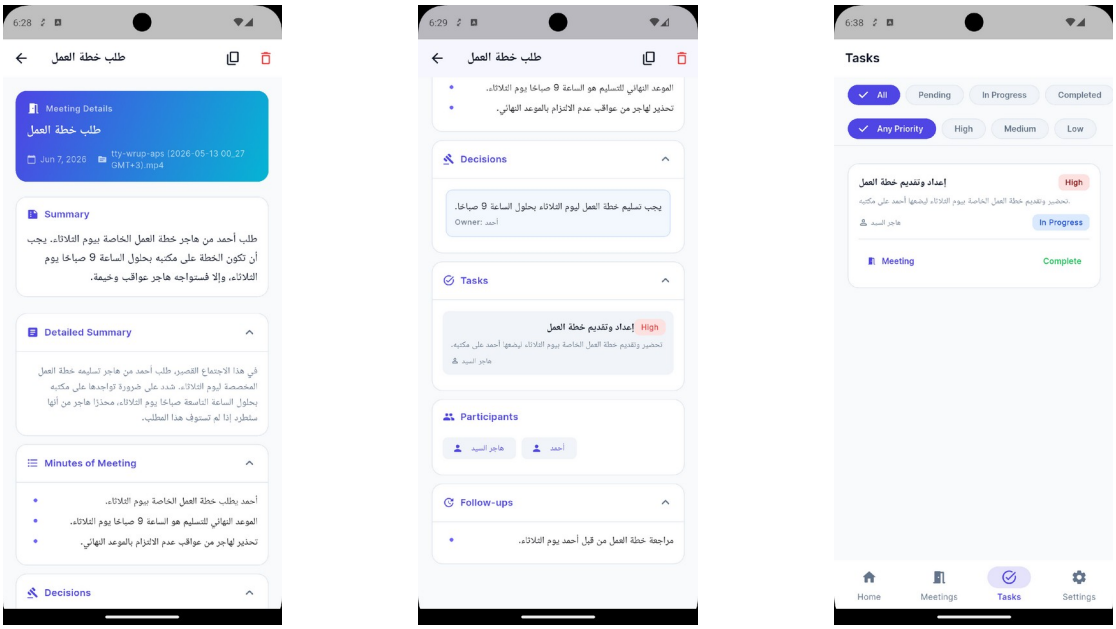


Figure 7.1: Testing and evaluation results summary.



Generated summary *Decisions/tasks* *Task tracking*

Figure 7.2: Result evidence screenshots for generated meeting output and task tracking.

7.5 Validation Against Objectives

Objective	How it was met	Evidence	Status
O1	Email/password auth, reset, persistent session, per-user isolation	TC-01–TC-03	Met
O2	File and link import with validation and invite-link rejection	TC-04, TC-06, TC-07	Met
O3	Structured JSON documentation with language preservation	TC-04, TC-05	Met
O4	Persistence, history, details, and task tracking	TC-09, TC-10	Met

8. Discussion After Applying the Solution

8.1 Problem Status After Solution

After applying MeetFlow AI, the manual, error-prone steps of producing meeting documentation are largely automated. Instead of re-listening to a recording and hand-writing minutes, a user obtains a structured summary, a list of decisions, and trackable tasks within a single mobile workflow. The weak link between decisions and follow-up — a central part of the original problem — is addressed by turning action items into a persistent, status-tracked task list. The problem is therefore substantially mitigated for the supported scenarios, though not entirely eliminated (see 8.3).

8.2 Benefits Achieved

- **Time saved:** minutes and action items are generated automatically rather than written by hand.
- **Consistency:** every meeting is documented in the same structured format.
- **Accountability:** decisions carry owners and tasks carry status, improving follow-through.
- **Inclusivity:** language preservation makes the tool useful for Arabic as well as English meetings.
- **Low cost & privacy:** the app runs on free/managed tiers and stores only metadata and results, not raw recordings.
- **Engineering quality:** a clean, layered, testable architecture with the AI capability cleanly abstracted.

8.3 Remaining Limitations

- **File-size ceiling:** direct client-to-model processing caps practical file size (50 MB), limiting very long recordings.
- **No live capture:** the app processes existing recordings/links, not live meetings.
- **AI variability:** output quality depends on audio clarity and the model, and may occasionally need correction.

Security rules / backend: production deployment should finalize Firestore rules and consider a backend proxy, while the MVP already keeps the Gemini API key out of the Flutter client by using Firebase AI Logic.

- **Single-user scope:** no team collaboration or sharing across accounts in the MVP.

8.4 Lessons Learned

- **Prompt engineering matters:** a strict schema and explicit language rules are essential to obtain reliable, parseable AI output.
- **Abstraction pays off:** hiding the AI behind an interface keeps the door open for a future backend without rewrites.
- **Defensive integration:** treating AI and network calls as fallible — with draft/failed states and humanized errors — greatly improves robustness.
- **Architecture discipline:** a feature-first clean architecture kept the solo codebase organized and easy to extend.
- **Security first:** designing the key-handling strategy early (Firebase AI Logic) avoided a costly redesign later.

8.5 Comparison: Before vs. After

Criterion	Before (manual)	After (MeetFlow AI)
Minutes creation	Manual, 30–60 min per meeting	Automatic, generated in minutes
Consistency	Varies by author	Uniform structured format
Action items	Scattered or lost	Trackable task list with status
Language support	English-centric tools	Preserves meeting language (incl. Arabic)
Cost	Paid tools or manual effort	Free/managed tiers
Data privacy	Raw recordings on vendor clouds	Only metadata/results stored

9. Conclusion and Future Work

9.1 Conclusion

This project set out to solve a common and costly problem: turning meetings into structured, actionable records. The result, MeetFlow AI, is a cross-platform mobile application that imports meeting audio, video, or links and uses a multimodal large language model to generate summaries, minutes, decisions, participants, follow-ups, and trackable tasks, all persisted securely per user. A working prototype demonstrates the full workflow and meets the four objectives defined at the outset. The project shows that, with modern multimodal AI and managed cloud services, a single developer can deliver a useful, secure, and low-cost intelligent application without a custom backend.

9.2 Contributions

- **An end-to-end mobile solution** that converts raw meeting media into structured, trackable documentation.
- **A practical integration pattern** for multimodal LLMs in a mobile app — strict JSON prompting, defensive parsing, and language preservation — with the AI key kept off the client via Firebase AI Logic.
- **A clean, feature-first reference architecture** in Flutter/Riverpod with the AI capability abstracted for a future backend.
- **A privacy-conscious data design** that stores only metadata and results and isolates data per user.

9.3 Future Work

- **Backend proxy & security rules:** move the AI call behind a server and finalize Firestore security rules for production.
- **Larger / live meetings:** support chunked or streamed processing and, eventually, live transcription.
- **Collaboration:** shared meetings, assigning tasks to other users, and team workspaces.
- **Calendar & notifications:** integrate with calendars and send reminders for due tasks.
- **Editing & export:** let users edit AI output and export to PDF/Docs or task tools.
- **Automated tests & analytics:** add unit/integration test suites and usage analytics to measure quality.

References

1. Flutter. (n.d.). Flutter: Build apps for any screen. Retrieved June 7, 2026, from https://flutter.dev/
2. Firebase. (n.d.). Welcome to Firebase for Flutter. Retrieved June 7, 2026, from https://firebase.google.com/docs/flutter
3. Firebase. (n.d.). Get Started with Firebase Authentication on Flutter. Retrieved June 7, 2026, from https://firebase.google.com/docs/auth/flutter/start
4. Firebase. (n.d.). Get started with Cloud Firestore. Retrieved June 7, 2026, from https://firebase.google.com/docs/firestore/quickstart
5. Firebase. (n.d.). Gemini API using Firebase AI Logic. Retrieved June 7, 2026, from https://firebase.google.com/docs/ai-logic
6. Google AI for Developers. (n.d.). Gemini models. Retrieved June 7, 2026, from https://ai.google.dev/gemini-api/docs/models/gemini-v2
7. Riverpod. (n.d.). Riverpod documentation. Retrieved June 7, 2026, from https://riverpod.dev/
8. Flutter team. (n.d.). go_router package. Retrieved June 7, 2026, from https://pub.dev/packages/go_router
9. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. Advances in Neural Information Processing Systems, 30.
10. Lewis, M., Liu, Y., Goyal, N., Ghazvininejad, M., Mohamed, A., Levy, O., Stoyanov, V., & Zettlemoyer, L. (2020). BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. Proceedings of ACL 2020.
11. Zhang, J., Zhao, Y., Saleh, M., & Liu, P. J. (2020). PEGASUS: Pre-training with extracted gap-sentences for abstractive summarization. Proceedings of ICML 2020.
12. Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2023). Robust speech recognition via large-scale weak supervision. Proceedings of ICML 2023.

Appendices

Appendix A: Additional Diagrams

The main architecture, sequence, component, use case, and data-model diagrams are included in Chapters 4 and 5. No additional large diagrams are required for this submission.

Appendix B: Source Code Samples

The AI is instructed to return the following JSON structure (abbreviated), which the application parses into its data model:

```
{
 "title": "", "shortSummary": "", "detailedSummary": "",
 "minutesOfMeeting": [], "participants": [], "followUps": [],
 "decisions": [ { "text": "", "owner": "" } ],
 "tasks": [ { "title": "", "description": "", "assignee": "",
          "dueDate": "", "priority": "low|medium|high",
          "status": "pending" } ]
}
```

No additional long code listing is included here. The full source code is available in the project repository; the schema extract above is the most relevant code-level contract for the AI integration.

Appendix C: User Manual / Installation Guide

Setup and run instructions are provided in Section 6.5. For end users: install the app, register an account, open Import, choose a file or paste a link, set a title, and start processing; the generated summary, decisions, and tasks appear in the meeting details, and action items can be tracked in the Tasks tab.

Step-by-step app screenshots are provided in Figure 5.3. They show the login/register flow, home dashboard, upload and link import screens, processing state, generated meeting details, task tracking, and settings/theme screens.

Appendix D: Additional Test Cases

ID	Scenario	Expected Result	Actual Result	Status
TC-11	Password reset email	Reset email sent for a valid account	Password reset page sends reset email for valid accounts.	Pass
TC-12	Persisted session after restart	User remains logged in	Firebase auth stream restores the authenticated session on launch.	Pass
TC-13	Import from a valid public text link	Documentation generated from fetched text	Public text/link import path fetches content and sends it to AI.	Pass
TC-14	Change task priority	Priority updates and persists	Task priority is generated and displayed consistently.	Pass
TC-15	Toggle dark mode	Theme switches and persists	Theme toggle verified by light and dark settings screenshots.	Pass